Pydantic vs. TypedDict

A Comprehensive Interview and Practical Implementation Reference Guide

1. High-Level Core Difference

The single most critical distinction to memorize for interviews is how they handle data types:

- **TypedDict (Static Type Hinting):** It is a development-time tool. It checks structures *before* code runs using static analysis tools like MyPy or IDEs. At runtime, a `TypedDict` is an ordinary, plain Python dictionary. It does **not** perform data validation, modification, or coercion when your program executes.
- **Pydantic (Runtime Validation & Coercion):** It is an execution-time engine. While it supports static typing, its primary job happens when code runs. It actively inspects incoming data, parses it, forces data into specified types (coercion), throws explicit validation errors if data doesn't conform, and outputs structured class instances.

> 🔥 **Interview Cheat-Sheet Metric:** If you pass the string `"123"` into a field typed as `int`:
> - `TypedDict` accepts it without a peep at runtime, which can crash downstream math calculations.
> - `Pydantic` instantly converts (coerces) it into the actual integer `123`.

2. Feature Comparison Matrix

Feature	TypedDict (Standard Library)	Pydantic (BaseModel)
Underlying Object	Standard Python `dict`	Custom Pydantic Class Instance
When it checks	Static analysis time (IDE / MyPy)	Runtime (during data instantiation)
Data Coercion	❌ None. Accepts mismatched values.	✅ Automatic (e.g., `"5"` becomes `5`)
Error Handling	❌ No runtime errors for wrong types	✅ Raises `ValidationError` on mismatch
Default Values	❌ Not natively supported inside the class	✅ Fully supported via assignments or `Field`
Handling Missing Keys	Uses `total=False` or `NotRequired[]`	Handled via `Optional[]` and default values
Performance Overhead	🚀 Zero overhead at runtime	⚡ Small parsing overhead per instantiation

3. Structural Differences & Handling Missing Fields

When implementing APIs or Large Language Model (LLM) extraction layers, dealing with missing or alternative schemas is a daily reality. Here is how both handle structural definitions and missing attributes.

TypedDict Behavior

By default, a `TypedDict` expects every declared key to exist. If a key is missing, static checkers report an error. To make keys structurally optional to omit entirely, you must leverage `total=False` or the `NotRequired` wrapper.

Crucially, writing `Optional[list[str]]` inside a `TypedDict` does **not** make the key optional; it merely means that *if* the key exists, its value can be `None`.

Pydantic Behavior

Pydantic relies on assignment defaults to figure out if a field is required. If a field lacks a default value, it is strictly mandatory. If you want a field to be omittable, you simply assign a default value like `None` or use `Field(default=...)`.

4. Practical Code Implementations

The following implementations showcase how both structures behave with identical dirty input data, showing runtime actions.

Implementation A: The TypedDict Approach

```
from typing import TypedDict, Annotated, Optional
from typing_extensions import NotRequired  # Pre-3.11 compatibility

class ReviewDict(TypedDict):
    key_themes: Annotated[list[str], "Themes discussed in the review"]
    summary: Annotated[str, "A brief summary"]
    sentiment: str
    # NotRequired allows the key to be missing entirely from the dictionary
    pros: NotRequired[Annotated[Optional[list[str]], "List of pros"]]
    cons: NotRequired[Annotated[Optional[list[str]], "List of cons"]]

# Clean data matches perfectly
clean_data = {
    "key_themes": ["Pricing"],
    "summary": "Too expensive",
    "sentiment": "Negative"
}
review_1: ReviewDict = clean_data  # Validated by IDE static analyzers

# DIRTY RUNTIME DATA: Type mismatch and missing fields
dirty_data = {
    "key_themes": "Not a list, just a string!",  # Type violation
    "summary": 1004,                             # Type violation (int instead of str)
    "sentiment": "Negative"
    # 'pros' and 'cons' are missing completely (allowed via NotRequired)
}

# RUNTIME CONSEQUENCE:
review_2: ReviewDict = dirty_data
print("TypedDict Runtime Output:", review_2)
# Output remains exactly as dirty_data. No errors are raised!
# review_2["summary"] + " text" will crash your application later at runtime.
```

Implementation B: The Pydantic Approach

```
from pydantic import BaseModel, Field, ValidationError
from typing import Optional

class ReviewModel(BaseModel):
    key_themes: list[str] = Field(description="Themes discussed in the review")
    summary: str = Field(description="A brief summary")
    sentiment: str
    # Providing a default value makes the field optional during parsing
    pros: Optional[list[str]] = Field(default=None, description="List of pros")
    cons: Optional[list[str]] = Field(default=None, description="List of cons")

# Test 1: Data Coercion and Default Value Fallback
semi_dirty_data = {
    "key_themes": ["UI/UX"],
    "summary": 2026,          # Integer instead of string!
    "sentiment": "Positive"   # pros and cons are omitted completely
}

try:
    obj = ReviewModel(**semi_dirty_data)
    print("Pydantic Coercion Output:")
    print("summary type:", type(obj.summary), "value:", obj.summary)
    print("pros value:", obj.pros)
except ValidationError as e:
    print(e)

# Test 2: Outright Validation Breakdown
broken_data = {
    "key_themes": "This is completely broken, not a list",
    "summary": "Great tool",
    "sentiment": "Positive"
}

print("
Pydantic Validation Catch:")
try:
    ReviewModel(**broken_data)
except ValidationError as e:
    print(e.json(indent=2))  # Automatically catches structural failures
```

5. Core Architectural Rules for Production

When to choose TypedDict:

- **JSON/Dictionary Interoperability:** When you are passing structures directly into legacy functions or third-party libraries that explicitly expect a pure Python `dict` subclass and cannot handle external class models.
- **Performance Critical Hotpaths:** If you are processing millions of messages per second and doing manual checks elsewhere, the parsing overhead of Pydantic might slow down processing speeds.

When to choose Pydantic:

- **Boundary Untrusted Input (APIs / Webhooks / LLMs):** Use it whenever data enters your application from an external, untrusted environment. It guarantees structural and logical purity before processing starts.
- **Complex Parsing Tasks:** When you need to transform inputs dynamically (e.g., parsing ISO strings into Python datetime objects, or validating custom patterns using regular expressions via field validators).
- **Structured LLM Output Generation:** Modern AI orchestration layers heavily favor Pydantic because its schemas can be seamlessly transformed into raw JSON-schema objects, guiding models to output structured extractions.